

Process and Thread Switching

Overhead and Performance Evaluation

Prashant Jadav

Fundamentals of Operating System Design (FOSD)

Learning Objectives

By the end of this topic, students should be able to:

- Explain process switching and thread switching.
- Identify the major sources of context-switch overhead.
- Distinguish process-switch overhead from thread-switch overhead.
- Calculate context-switch overhead using quantitative measures.
- Evaluate the effect of switching overhead on CPU utilization and throughput.
- Analyze the trade-off between responsiveness and switching cost.

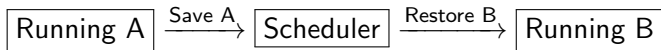
What Is a Context Switch?

Definition

A **context switch** is the operation of saving the execution state of the currently running execution entity and restoring the saved state of another entity.

The execution context may contain:

- Program Counter (PC)
- Stack Pointer (SP)
- General-purpose registers
- Processor status / flags
- Scheduling information
- Address-space information when required



Why Does Context Switching Have Overhead?

A context switch performs useful system-management work, but it does not directly execute the user's application instructions.

Typical overhead sources:

- ① Saving CPU registers.
- ② Updating process/thread state.
- ③ Selecting the next runnable entity.
- ④ Restoring CPU registers.
- ⑤ Updating scheduling queues.
- ⑥ Changing address-space context when necessary.
- ⑦ Cache and TLB effects.
- ⑧ Synchronization inside the kernel.

Key idea

A context switch consumes CPU time and can also reduce cache/TLB locality, producing indirect performance costs.

Process Switching

Process switch

A process switch changes CPU execution from a thread/process belonging to one process to an execution context belonging to another process.

Potential operations

- Save current CPU context.
- Load the new CPU context.
- Change process/thread state.
- Change address-space context.
- Update memory-management state.
- Potentially affect TLB and cache locality.

$P_1 \rightarrow \text{Context Save} \rightarrow \text{Address Space Change} \rightarrow \text{Context Restore} \rightarrow P_2$

Thread Switching

Thread switch

A thread switch changes execution from one thread to another. If both threads belong to the same process, their address space is normally shared.

Typical operations

- Save current thread context.
- Update thread state.
- Select another runnable thread.
- Restore the new thread context.
- Continue execution.

Important

A switch between threads of the same process can avoid some address-space changes, but the exact cost is OS- and hardware-dependent.

Process Switch vs. Thread Switch

Aspect	Thread switch	Process switch
CPU registers	Save/restore	Save/restore
Program counter	Changes	Changes
Stack pointer	Changes	Changes
Thread state	Changes	Changes
Address space	Usually unchanged for same process	Usually changes
Page-table context	Usually unchanged	May change
Cache/TLB locality	Can be affected	Can be affected, potentially more
Typical overhead	Often lower	Often higher

Caution

“Thread switch is always cheaper” is not universally true. Actual overhead depends on OS implementation, architecture and workload.

Direct and Indirect Switching Costs

Direct cost

- Time spent saving/restoring context.
- Scheduler execution.
- Kernel bookkeeping.

Indirect cost

- Cache misses after switching.
- TLB misses or address-translation effects.
- Branch-prediction disruption.
- Reduced locality of reference.
- Additional memory traffic.

Total performance impact

$$T_{\text{switch impact}} = T_{\text{direct}} + T_{\text{indirect}}$$

Basic Context-Switch Cost Model

Suppose:

- N = number of context switches
- t_{cs} = average time for one context switch

Then:

$$T_{\text{switch}} = N \times t_{cs}$$

If the total observation interval is T :

$$\text{Switch overhead fraction} = \frac{Nt_{cs}}{T}$$

Therefore:

$$\text{Useful CPU fraction} = 1 - \frac{Nt_{cs}}{T}$$

This simple model considers direct switching cost only.

Numerical Example 1: Context-Switch Overhead

A system performs **1,000 context switches per second**.

Average context-switch time:

$$t_{cs} = 5 \mu s$$

Calculate the CPU time consumed by context switching per second.

Solution

$$T_{\text{switch}} = 1000 \times 5 \mu s = 5000 \mu s = 5 \text{ ms}$$

Therefore:

$$\text{Overhead fraction} = \frac{5}{1000} = 0.005 = 0.5\%$$

Interpretation: About 0.5% of CPU time is consumed by the direct switching cost.

Numerical Example 2: Higher Switching Frequency

Suppose:

$$t_{cs} = 8 \mu s$$

Compare two scheduling configurations.

	Configuration A	Configuration B
Context switches/sec	500	5000
Switch time	$8 \mu s$	$8 \mu s$
Time spent switching/sec	$4 ms$	$40 ms$
Direct overhead	0.4%	4%

Observation

Increasing switching frequency by $10\times$ increases the direct switching overhead by $10\times$, assuming the switching cost remains constant.

Context Switching and Round Robin

In Round Robin scheduling, the time quantum q affects switching frequency.
If processes frequently use the entire quantum:

$$\text{Switch frequency} \approx \frac{1}{q}$$

Ignoring other costs, the direct overhead fraction can be approximated by:

$$\text{Overhead} \approx \frac{t_{cs}}{q + t_{cs}}$$

where:

- q = time quantum
- t_{cs} = context-switch time

Trade-off

Smaller q improves responsiveness but can increase switching overhead.

Numerical Example 3: Time Quantum

Given:

$$q = 10 \text{ ms}, \quad t_{cs} = 1 \text{ ms}$$

Approximate the direct switching overhead:

$$\text{Overhead} = \frac{1}{10 + 1} = \frac{1}{11} \approx 9.09\%$$

Now increase the time quantum:

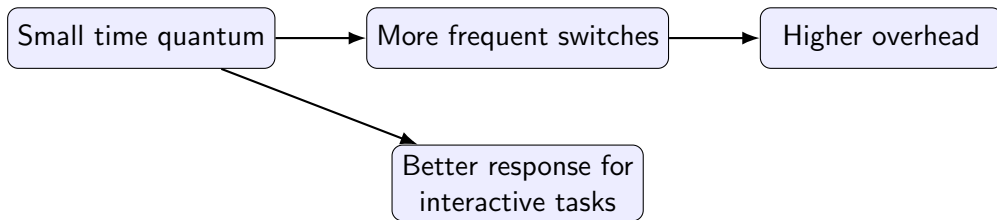
$$q = 50 \text{ ms}$$

$$\text{Overhead} = \frac{1}{50 + 1} = \frac{1}{51} \approx 1.96\%$$

Interpretation

A larger quantum reduces direct context-switch overhead, but it may increase the waiting/response time experienced by interactive tasks.

Responsiveness vs. Switching Overhead



Scheduling design problem

The time quantum should be selected considering workload characteristics, response-time requirements and switching overhead.

Cache and TLB Effects

Context switching can create performance costs beyond the direct switching time.

Cache effect

- The new thread/process may access different instructions and data.
- Existing cache contents may become less useful.
- Cache misses can increase memory-access latency.

TLB effect

- A process switch may involve a different address space.
- Translation entries may become less useful or require replacement.
- Modern hardware can reduce this cost using mechanisms such as address-space tags.

Important

The exact effect depends strongly on the processor architecture and OS implementation.

Performance Metrics for Evaluating Switching

Context-switch performance should not be evaluated using switching time alone.

- **Context-switch latency:** Time required for the switch.
- **Switches per second:** Frequency of switching.
- **CPU overhead:** Fraction of CPU time spent switching.
- **Throughput:** Completed work per unit time.
- **Response time:** Time until a task receives service.
- **Turnaround time:** Time from submission to completion.
- **Cache/TLB miss rate:** Indicator of indirect locality cost.
- **CPU utilization:** Fraction of time doing useful work.

Performance Evaluation Model

Suppose a workload performs:

N context switches

with average direct cost:

t_{cs}

and total useful execution time:

T_u

Then:

$$T_{\text{total}} = T_u + Nt_{cs} + T_{\text{indirect}}$$

where T_{indirect} represents cache, TLB, branch-prediction and related effects.

Therefore:

$$\text{CPU efficiency} = \frac{T_u}{T_u + Nt_{cs} + T_{\text{indirect}}}$$

This gives a more realistic conceptual model of switching efficiency.

Numerical Example 4: CPU Efficiency

Consider:

$$T_u = 950 \text{ ms}$$

$$N = 10,000$$

$$t_{cs} = 2 \mu s$$

Ignoring indirect effects:

$$T_{\text{switch}} = 10,000 \times 2 \mu s = 20 \text{ ms}$$

Thus:

$$T_{\text{total}} = 950 + 20 = 970 \text{ ms}$$

CPU efficiency:

$$\frac{950}{970} \approx 97.94\%$$

Interpretation

Approximately 2.06% of the observed CPU time is consumed by direct switching overhead.

Factors Affecting Context-Switch Cost

Hardware factors

- CPU architecture
- Number of registers
- TLB design
- Cache hierarchy
- Hardware address-space support
- Memory latency

Software factors

- Kernel implementation
- Scheduler design
- Process/thread model
- Synchronization
- Address-space management
- Kernel data structures

Conclusion

Context-switch time is not a universal constant; it must be measured for the specific OS, hardware and workload.

How to Measure Context-Switch Performance

A practical evaluation can measure:

- 1 Timestamp immediately before switching.
- 2 Perform or trigger a controlled context switch.
- 3 Timestamp after the new execution context starts.
- 4 Repeat many times.
- 5 Calculate average, median and percentile latency.

Useful experimental metrics

$$\text{Average latency} = \frac{\sum_{i=1}^N t_i}{N}$$

Also consider:

$P50$, $P95$, $P99$

Why multiple statistics?

Average latency alone can hide occasional expensive switches caused by interrupts, cache effects, contention or system activity.

Benchmark Design Considerations

A meaningful switching benchmark should control or record:

- CPU frequency and power-management state.
- Number of CPU cores.
- Processor affinity.
- Background system activity.
- Number of runnable tasks.
- Process versus same-process thread switching.
- Memory-access patterns.
- Warm-up iterations.

Experimental principle

Compare measurements under the same hardware, OS configuration and workload conditions.

Process vs. Thread Switching: Analytical Scenario

Suppose:

- Process switch = $6\ \mu s$
- Same-process thread switch = $2\ \mu s$
- Workload performs 20,000 switches per second.

Direct switching time:

$$T_P = 20,000(6\ \mu s) = 120\ ms/s$$

$$T_T = 20,000(2\ \mu s) = 40\ ms/s$$

Therefore, direct overhead is:

12% versus 4%

Question

What additional measurements would you collect before concluding that the thread-based design is faster overall?

Important Performance Trade-offs

- **Smaller time quantum**

→ better responsiveness → more switching → higher overhead

- **Larger time quantum**

→ fewer switches → lower direct overhead → potentially poorer response

- **More threads**

→ more concurrency → possible scheduling/synchronization overhead

- **More process isolation**

→ stronger isolation → potentially greater switching/address-space cost

Higher-Order Thinking Questions

- 1 A Round Robin scheduler reduces its time quantum from 20 *ms* to 5 *ms*. Explain the likely effect on responsiveness, context-switch frequency and CPU overhead.
- 2 Two systems have the same context-switch latency, but System A performs twice as many switches per second. Explain why its CPU efficiency may be lower.
- 3 A benchmark reports a low context-switch time but poor application performance after switching. Identify possible indirect causes.
- 4 Why is it insufficient to compare only the average context-switch latency when evaluating two OS implementations?

Numerical Practice Question

A CPU executes a workload for 2 seconds.

During this period:

$$N = 50,000 \text{ context switches}$$

and:

$$t_{cs} = 3 \mu s$$

Calculate:

- 1 Total direct switching time.
- 2 Percentage of CPU time consumed by switching.
- 3 Useful CPU percentage, ignoring indirect effects.

Hint

$$T_{\text{switch}} = N \times t_{cs}$$

$$\text{Overhead} = \frac{T_{\text{switch}}}{2s} \times 100$$

Summary

Concept	Key idea
Context switch	Save one execution context and restore another.
Process switch	May involve changing address-space context.
Thread switch	Same-process thread switches can avoid some process-level changes.
Direct overhead	CPU time spent performing the switch and scheduler work.
Indirect overhead	Cache, TLB, branch-prediction and locality effects.
Switch frequency	Higher frequency increases cumulative switching cost.
Time quantum	Smaller quantum improves responsiveness but may increase overhead.
Performance evaluation	Use latency, overhead, throughput, response time and efficiency.

Key takeaway

Context switching is necessary for multitasking, but excessive switching can reduce the

The objective is not to eliminate context switches.

The objective is to achieve an appropriate balance among:

Responsiveness + Throughput + CPU Efficiency

Scheduling Policy + Context-Switch Cost + Workload Characteristics

Performance must be evaluated using measurements, not assumptions.